CONTRACT WOLF

Security Assessment

Ceitnot Utilities

Verified on 04/06/2026

SUMMARY

Project	CHAIN	METHODOLOGY
Ceitnot	Arbitrum	Manual & Automatic Analysis
FILES	**DELIVERY**	**TYPE**
Multiple	04/06/2026	Standard Audit

5	0	2	3	0	0	5
Total Findings	Critical	Major	Medium	Minor	Informational	Resolved

🟥 0 Critical	An exposure that can affect the contract functions in several events that can risk and disrupt the contract
🟧 2 Major	An opening & exposure to manipulate the contract in an unwanted manner
🟧 3 Medium	An opening that could affect the outcome in executing the contract in a specific situation
⬜ 0 Minor	An opening but doesn't have an impact on the functionality of the contract
🟦 0 Informational	An opening that consists information but will not risk or affect the contract
🟩 5 Resolved	ContractWolf's findings has been acknowledged & resolved by the project

STATUS ✔ **AUDIT PASSED**

TABLE OF CONTENTS | Ceitnot

Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

Project Information

Token/Project Details
Inheritance Graph
Call Graph

Findings

Issues
SWC Attacks
CW Assessment
Fixes & Recommendation
Audit Comments

DISCLAIMER | Ceitnot

ContractWolf audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

ContractWolf does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

ContractWolf provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

ContractWolf's presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

Each company or project should be liable to its security flaws and functionalities.

SCOPE OF WORK | Ceitnot

Ceitnot team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Ceitnot**.

AUDITING APPROACH | Ceitnot

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.
2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.
3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.
4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

TOKEN DETAILS | Ceitnot

Ceitnot is an EVM lending and CDP protocol: multi-market collateralised debt, stablecoin (aUSD), PSM, router, oracle integration, governance (token + vote-escrow + OpenZeppelin Governor + Timelock), and treasury flows.

Token Name	Symbol	Decimal	Total Supply	Chain
-	-	-	-	Arbitrum

SOURCE

Source	*Market Strategy : 0x070b9c6bdbffabefe02de23840069f15eb821c55* *Engine : 0xd2168f8429acb4796465b07ca6ecf192d9b41619* *Aura USD : 0xe1b1a3814c3f5f3cdfd85a63225f7d16ecdd6785* *Aura PSM : 0x330c36C9Fe280a7d9328165DB7ca78e59b119e12* *Aura Vault : 0x4889aD99f1A28cf87b5EaC39714b3D9D3fd393Ad* *Oracle Relay : 0x6964d7529e0B7e9331a3Cf471065C6a46D916eFa* *Router : 0x4f083ab27345f353e61f04988c8fefc76eacbb7d* *Treasury : 0xeec09a4ec6fabef4587195296f2d0a4404c7a947*

FINDINGS | Ceitnot

Total Findings	Critical	Major	Medium	Minor	Informational	Resolved
5	0	2	3	0	0	5

This report has been prepared to state the issues and vulnerabilities for Ceitnot through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

ID	Title	File & Line #	Severity	Status
Logic Error	Active Market with Zero/Default Values	AuraMarketRegistry	Major	Resolved
Logic Error	Clamping to Zero vs. Reverting	AuraPSM	Major	Resolved
SWC-124	Write to Arbitrary Storage Location	AuraUSD	Medium	Resolved
SWC-104	Unchecked Call Return Value	OracleRelay	Medium	Resolved
SWC-101	Integer Overflow/Underflow	AuraUSD	Medium	Resolved

SWC ATTACKS | Ceitnot

Smart Contract Weakness Classification and Test Cases

ID	Description	Status
SWC-100	Function Default Visibility	Passed
SWC-101	Integer Overflow and Underflow	Not Passed
SWC-102	Outdated Compiler Version	Passed
SWC-103	Floating Pragma	Passed
SWC-104	Unchecked Call Return Value	Not Passed
SWC-105	Unprotected Ether Withdrawal	Passed
SWC-106	Unprotected SELF DESTRUCT Instruction	Passed
SWC-107	Reentrancy	Passed
SWC-108	State Variable Default Visibility	Passed
SWC-109	Uninitialized Storage Pointer	Passed
SWC-110	Assert Violation	Passed
SWC-111	Use of Deprecated Solidity Functions	Passed
SWC-112	Delegate call to Untrusted Callee	Passed
SWC-113	DoS with Failed Call	Passed
SWC-114	Transaction Order Dependence	Passed
SWC-115	Authorization through tx.origin	Passed
SWC-116	Block values as a proxy for time	Passed
SWC-117	Signature Malleability	Passed
SWC-118	Incorrect Constructor Name	Passed
SWC-119	Shadowing State Variables	Passed
SWC-120	Weak Sources of Randomness from Chain Attributes	Passed
SWC-121	Missing Protection against Signature Replay Attacks	Passed
SWC-122	Lack of Proper Signature Verification	Passed

ID	Description	Status
SWC-123	Requirement Violation	Passed
SWC-124	Write to Arbitrary Storage Location	Not Passed
SWC-125	Incorrect Inheritance Order	Passed
SWC-126	Insufficient Gas Griefing	Passed
SWC-127	Arbitrary Jump with Function Type Variable	Passed
SWC-128	DoS With Block Gas Limit	Passed
SWC-129	Typographical Error	Passed
SWC-130	Right-To-Left-Override control character(U+202E)	Passed
SWC-131	Presence of unused variables	Passed
SWC-132	Unexpected Ether balance	Passed
SWC-133	Hash Collisions With Multiple Variable Arguments	Passed
SWC-134	Message call with hardcoded gas amount	Passed
SWC-135	Code With No Effects	Passed
SWC-136	Unencrypted Private Data On-Chain	Passed

CW ASSESSMENT | Ceitnot

ContractWolf Vulnerability and Security Tests

ID	Name	Description	Status
CW-001	Multiple Version	Presence of multiple compiler version across all contracts	✔
CW-002	Incorrect Access Control	Additional checks for critical logic and flow	✔
CW-003	Payable Contract	A function to withdraw ether should exist otherwise the ether will be trapped	✔
CW-004	Custom Modifier	major recheck for custom modifier logic	✔
CW-005	Divide Before Multiply	Performing multiplication before division is generally better to avoid loss of precision	✔
CW-006	Multiple Calls	Functions with multiple internal calls	✔
CW-007	Deprecated Keywords	Use of deprecated functions/operators such as block.blockhash() for blockhash(), msg.gas for gasleft(), throw for revert(), sha3() for keccak256(), callcode() for delegatecall(), suicide() for selfdestruct(), constant for view or var for actual type name should be avoided to prevent unintended errors with newer compiler versions	✔
CW-008	Unused Contract	Presence of an unused, unimported or uncalled contract	✔
CW-009	Assembly Usage	Use of EVM assembly is error-prone and should be avoided or double-checked for correctness	✔
CW-010	Similar Variable Names	Variables with similar names could be confused for each other and therefore should be avoided	✔
CW-011	Commented Code	Removal of commented/unused code lines	✔
CW-012	SafeMath Override	SafeMath is no longer needed starting with Solidity v0.8+. The compiler now has built-in overflow checking.	✔

FIXES & RECOMMENDATION

SWC-101 | Integer Overflow & Underflow

The internal function **`_deductBalance()`** reduces both **`balanceOf[from]`** and **`totalSupply`**. This function is called from **`transfer()`**, **`transferFrom()`**, **`burn()`**, and **`burnFrom()`**.

While **`burn()`** should reduce the total supply, **`transfer()`** must not change the total supply. The current implementation incorrectly decreases **`totalSupply`** on every token transfer, breaking the ERC-20 accounting.

Affected Code :

```
function _deductBalance(address from, uint256 amount) internal {
    if (balanceOf[from] < amount) revert //;
    unchecked {
        balanceOf[from] -= amount;
        totalSupply     -= amount;   // ❌ Should NOT happen in transfers
    }
}

function transfer(address to, uint256 amount) external returns (bool) {
    _deductBalance(msg.sender, amount);   // ❌ totalSupply reduced
    balanceOf[to] += amount;              // totalSupply already lowered
    //
}
```

Recommendation

Separate the burn and transfer logic. Create a `_transfer()` function that does not touch totalSupply.

```
function transfer(address to, uint256 amount) external returns (bool) {
    _transfer(msg.sender, to, amount);
    return true;
}

function transferFrom(address from, address to, uint256 amount) external returns
(bool) {
    uint256 allowed = allowance[from][msg.sender];
    if (allowed != type(uint256).max) {
        if (allowed < amount) revert AuraUSD__InsufficientAllowance();
        unchecked { allowance[from][msg.sender] = allowed - amount; }
    }
    _transfer(from, to, amount);
    return true;
}

function _transfer(address from, address to, uint256 amount) internal {
    if (balanceOf[from] < amount) revert AuraUSD__InsufficientBalance();
    unchecked {
        balanceOf[from] -= amount;
        balanceOf[to]   += amount;
    }
    emit Transfer(from, to, amount);
}

function _burn(address from, uint256 amount) internal {
    if (balanceOf[from] < amount) revert AuraUSD__InsufficientBalance();
    unchecked {
        balanceOf[from] -= amount;
        totalSupply     -= amount;
    }
    emit Transfer(from, address(0), amount);
}

// Then update burn() and burnFrom() to use _burn() instead of _deductBalance()
```

SWC-124 | Signature Malleability / EIP-2612 Nonce Handling

The nonce is incremented before the signature is validated. If the signature is invalid, the nonce is still consumed, rendering all future permits for that owner useless.

```
bytes32 structHash = keccak256(
    abi.encode(PERMIT_TYPEHASH, owner, spender, value, nonces[owner]++,
deadline)
);
```

Recommendation

Increment the nonce only after the signature recovery succeeds.

```
function permit() external {
    if (block.timestamp > deadline) revert AuraUSD__PermitExpired();

    uint256 currentNonce = nonces[owner];
    bytes32 structHash = keccak256(
        abi.encode(PERMIT_TYPEHASH, owner, spender, value, currentNonce,
deadline)
    );
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", DOMAIN_SEPARATOR(),
structHash));
    address recovered = ecrecover(digest, v, r, s);
    if (recovered == address(0) || recovered != owner) revert
AuraUSD__InvalidSignature();

    unchecked { nonces[owner] = currentNonce + 1; }
    allowance[owner][spender] = value;
    emit Approval(owner, spender, value);
}
```

LOGIC ERROR | Active Market with Zero/Default Values

The **`addMarket()`** function creates a market with **`isActive = true`** but leaves many critical risk parameters at their default 0 values, including:

- **closeFactorBps** *(liquidation close factor)*
- **fullLiquidationThresholdBps**
- **protocolLiquidationFeeBps**
- **yieldFeeBps**, **originationFeeBps**
- **debtCeiling**, **auctionDuration**, etc.
-

If the protocol's engine does not explicitly validate these fields, a market may be considered active while having unsafe defaults (e.g., closeFactorBps = 0 might disable liquidations, or debtCeiling = 0 might block CDP minting).

Recommendation

Either require all essential parameters in **`addMarket()`** or add a separate **`activateMarket()`** that enforces non-zero values for critical fields.

```
function addMarket(
    address vault,
    address oracle,
    uint16  ltvBps,
    uint16  liquidationThresholdBps,
    uint16  liquidationPenaltyBps,
    uint256 supplyCap,
    uint256 borrowCap,
    bool    isIsolated,
    uint256 isolatedBorrowCap,
    uint16  closeFactorBps,                // new param
    uint16  fullLiquidationThresholdBps,   // new param
    uint16  protocolLiquidationFeeBps,     // new param
    uint16  yieldFeeBps,                   // new param
    uint16  originationFeeBps,             // new param
    uint256 debtCeiling                    // new param
) external onlyAdmin returns (uint256 marketId) {
    //
    if (closeFactorBps == 0 || fullLiquidationThresholdBps == 0) revert
Registry__InvalidParams();
    //
}
```

Alternatively fix : require that **`activateMarket()`** checks non-zero values before setting **`isActive = true`**.

SWC-104 | Unchecked Return Value / Revert on Non-Chainlink Feeds

The function **`_tryChainlink()`** calls **`decimals()`** on the feed without a **`try-catch`**. If the feed does not implement **`decimals()`** (e.g., a misconfigured address or a different oracle type), the whole transaction will revert, preventing the fallback feed from being used.

```
uint8 feedDecimals = IChainlinkAggregator(feed).decimals();
```

Recommendation

Wrap the **`decimals()`** call in a **`try-catch`** and treat any failure as invalid data.

```
function _tryChainlink(address feed) internal view returns (bool ok, uint256
value, uint256 timestamp) {
    if (feed == address(0)) return (false, 0, 0);
    try IChainlinkAggregator(feed).latestRoundData() returns (
        uint80, int256 answer, uint256, uint256 updatedAt, uint80
    ) {
        if (answer <= 0) return (true, 0, updatedAt);
        uint8 feedDecimals;
        try IChainlinkAggregator(feed).decimals() returns (uint8 d) {
            feedDecimals = d;
        } catch {
            return (false, 0, 0);
        }
        uint256 price = uint256(answer);
        if (feedDecimals < 18) {
            price = price * (10 ** (18 - feedDecimals));
        } else if (feedDecimals > 18) {
            price = price / (10 ** (feedDecimals - 18));
        }
        return (true, price, updatedAt);
    } catch {
        return (false, 0, 0);
    }
}
```

LOGIC ERROR | Clamping to Zero vs. Reverting

When burning more **aUSD** than the **PSM** has ever minted, the code clamps **`mintedViaPsm`** to zero instead of reverting. This can lead to a situation where **`mintedViaPsm`** does not accurately reflect net issuance, though the reserves check (available - *feeReserves* < *stableOut*) still prevents theft.

```
mintedViaPsm = mintedViaPsm >= amount ? mintedViaPsm - amount : 0;
```

Recommendation

Revert if *mintedViaPsm* < *amount* to maintain strict accounting.

```
if (mintedViaPsm < amount) revert PSM__InsufficientMinted();
unchecked { mintedViaPsm -= amount; }
```

AUDIT COMMENTS | Ceitnot

Smart Contract audit comment for a non-technical perspective

- 🟢 All issues has been ACKNOWLEDGED & FIXED
- 🟢 Contract cannot be paused
- 🟢 Owner can renounce and transfer ownership
- 🟢 Owner can burn tokens
- 🟢 Owner cannot mint after initial deployment
- 🟢 Owner cannot set max transaction limit
- 🟢 Owner cannot block users

CONTRACTWOLF

Blockchain Security - Smart Contract Audits